

Lista zadań nr 11

Zadanie 1. (2 pkt)

Rozszerz język z wykładu o alternatywę ($e_1 \mid\mid e_2$), która podobnie jak zdefiniowana na wykładzie koniunkcja nie będzie obliczała drugiego argumentu, jeśli wynik pierwszego jest wystarczający do określenia wartości całego wyrażenia.

Zadanie 2. (2 pkt)

Gdy zaimplementowana na wykładzie funkcja `compile_cond` napotka wyrażenie warunkowe, lub let-wyrażenie, to przechodzi do domyślnej implementacji. Zademonstruj, że kod generowany w takim przypadku jest przesadnie skomplikowany (np. dla wyrażenia `if if x then a > b else a < b then 42 else 13`). Następnie zmodyfikuj funkcję `compile_cond` tak, aby w przypadku wyrażeń warunkowych i let-wyrażeń wywoływała się rekurencyjnie na (niektórych) podwyrażeniach by uniknąć generowania niepotrzebnych skoków.

Zadanie 3. (1 pkt)

Rozszerz implementację kompilatora o obsługę wyrażenia postaci $x := e$, które będzie przypisywało wartość e do (istniejącej) już zmiennej x . Użyj instrukcji `MSetLocal n` która zapisuje wartość akumulatora do lokalnej zmiennej znajdującej się na n -tej pozycji na stosie.

Zadanie 4. (2 pkt)

Rozszerz implementację kompilatora o pętlę `while  $e_1$  do  $e_2$  done`. Jeśli nie rozwiązałeś poprzedniego zadania, to ciężko będzie napisać sensowny program korzystający z nowej konstrukcji (dlaczego?).

Zadanie 5. (2 pkt)

Rozszerz implementację kompilatora o obsługę funkcji rekurencyjnych, np. wprowadzanych konstrukcją `let rec f x = e1 in e2`. Podobnie jak na wykładzie, załóż, że funkcje muszą być zamknięte, tj. nie korzystać z żadnych zmiennych z otoczenia (poza sobą samą i swoimi argumentami).

Wskazówka: zmodyfikuj implementację środowisk tak, aby informacja o zmiennej była jedną z dwóch możliwości: albo jest to zwykła zmienna żyjąca na stosie, albo jest to funkcja, reprezentowana przez etykietę.

Zadanie 6. (2 pkt)

Rozszerz implementację kompilatora o obsługę funkcji wieloargumentowych. Zastanów się, czy nie warto zmienić składni konkretnej tak, aby nie było niejednoznaczności, np. `f x y` czy ma być traktowane jako `(f x) y` czy wywołanie funkcji z dwoma argumentami?

Zadanie 7. (1 pkt)

Zauważ, że w przypadku wołania funkcji na ogonowej pozycji, można użyć wskaźnika powrotu zachowanego przy wywołaniu bieżącej funkcji, przez co zamiast instrukcji wołania funkcji, można użyć instrukcji skoku. Taka optymalizacja nazywa się *optymalizacją rekursji ogonowej* i pozwala uniknąć problemu przepełnienia stosu w przypadku głębokiej rekurencji. Nie jest jednak to takie proste, bo trzeba jeszcze zadbać o to, by w momencie skoku stos był w odpowiednim stanie (np. by nie zawierał zmiennych lokalnych, które już nie są potrzebne). Zaimplementuj funkcję `compile_tail`, która kompiluje wyrażenia na ogonowej pozycji, która w przypadku wywołania funkcji generuje instrukcję skoku. Np. dla programu

```
let f = fun x -> let g = fun x -> x in g (g x) in f 42
```

wzorcowe rozwiązanie może wygenerować poniższy kod.

```
MGetLabel $L_2
MPush
MConst 42
MPush
MGetLocal 1
```

```
    MCallAcc
    MPopN      1
    MPopN      1
    MJump      $L_0
$L_2
    MGetLabel $L_1
    MPush
    MGetLocal 1
    MPush
    MGetLocal 1
    MCallAcc
    MPopN      1
    MPush
    MGetLocal 1
    MPush
    MGetLocal 1
    MSetLocal 3
    MPopAcc
    MPopN      2
    MJumpAcc
$L_1
    MGetLocal 0
    MRet
$L_0
```